

RAG-Augmented Multi-Agent LLM Framework for Explainable Software Vulnerability Detection Using CWE Knowledge Bases

Richa Verma (25MCSS02) Advisor: Dr. Akshay Pandey

A small Java practice set and a first check

Where this fits

The aim is a Java tool that explains a weakness in plain words, using a CWE name. This note is only about the first files and a very simple checker.

Nothing here is a public benchmark. Juliet and OWASP are larger test sets used by other people. We have not used them yet.

The idea we started with

Before any model, we needed a set of files whose correct answers we already knew. If we cannot score a tiny set, we cannot trust a score on a large set.

The set had to contain both unsafe and safe code. A checker that flags everything looks perfect if every file is unsafe. Safe files are what reveal false alarms.

Why we chose this way

We wrote the files ourselves, two per weakness, instead of downloading Juliet on day one. We wanted the labels, the wording, and the split to be under our control while the plumbing was new.

We split 8 files for practice and 4 files held back. The held-back files are not a serious test. They only stop us from changing the patterns after seeing every example.

The first checker is a list of text patterns, not a program analyzer. We chose that on purpose. It is easy to explain, easy to score, and easy to beat later. If a later model cannot beat this simple checker, the model is not earning its cost.

What we actually did

We wrote 12 short Java files, two for each of the six weaknesses: one unsafe and one safe. For example, one file builds a database query by sticking user text onto the query, and the paired file uses a safer call.

Eight files are the practice split. Four files are held back. The held-back split has two unsafe files and two safe files. We do not tune the checker on those four.

The checker looks for risky text patterns. It might see a query built with string joining, or a password written as a literal. It does not understand what the program does.

On these 12 files it marked every unsafe file and did not mark a safe file. Precision means how many alarms were real bugs. Recall means how many real bugs we caught. Both are 1.000, so F1 is 1.000. F1 is one number that combines those two.

A perfect score is easy to get when we wrote both the files and the patterns. It only shows that the checker matches this tiny set.

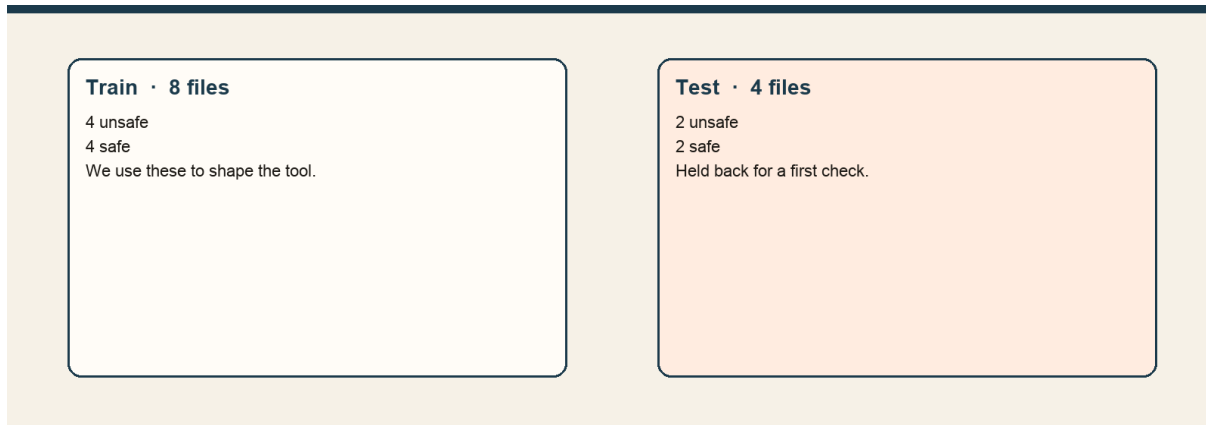
A concrete example

One practice file builds a SQL string with the username inside it. The checker sees that pattern and marks the file unsafe. The paired file uses a prepared statement. The checker does not mark it.

The four held-back files are the same idea for two other weaknesses: a password written in the source, and a weak hash. We did not change the patterns after seeing those four. They still came out right.

That is why the table is balanced: every weakness in this set has one unsafe file and one safe file. The score cannot hide behind a set that is almost all unsafe.

Picture



Eight files are for practice. Four files are held back for a first check.

The numbers

Split	Files	Unsafe	Safe
Train	8	4	4
Test	4	2	2
All	12	6	6

The score on all 12 files is precision 1.000, recall 1.000, F1 1.000. False alarms: 0. Missed bugs: 0.

What we concluded, and why

The checker can say that a pattern fired. It still cannot explain the weakness in CWE words, because those words are not stored yet.

The next piece of work is a small book of CWE facts: the name, what goes wrong, and a fix. The conclusion here is narrow: the pattern list matches the 12 files we wrote. It does not yet explain them.

What we are not saying

This perfect score is not evidence about real projects. These 12 files are not Juliet and not the OWASP Benchmark.